

第 21 章

支持向量机

支持向量机 (Support Vector Machine, SVM) 是一种二分类模型, 目标是寻找一个标准 (称为超平面) 对样本数据进行分割, 分割的原则是确保分类最优化 (类别之间的间隔最大)。当数据集较小时, 使用支持向量机进行分类非常有效。支持向量机是最好的现成分类器之一, 这里所谓的“现成”是指分类器不加修改即可直接使用。

在对原始数据分类的过程中, 可能无法使用线性方法实现分割。支持向量机在分类时, 把无法线性分割的数据映射到高维空间, 然后在高维空间找到分类最优的线性分类器。

Python 提供了不同的实现支持向量机的库 (例如 `sk-learn` 库、`LIBSVM` 库等), `OpenCV` 也提供了对支持向量机的支持, 对于上述库, 基本都可以直接使用, 无须深入了解支持向量机的原理。

21.1 理论基础

本节对支持向量机的基本原理做简单的介绍。

1. 分类

某 IT 企业在 2017 年通过笔试、面试的形式招聘了一批员工。2018 年, 企业针对这批员工在过去一年的实际表现进行了测评, 将他们的实际表现分别确定为 A 级 (优秀) 和 B 级 (良好)。这批员工的笔试成绩、面试成绩和等级如图 21-1 所示, 图中横坐标是笔试成绩, 纵坐标是面试成绩, 位于右上角的圆点表示测评成绩是 A 级, 位于左下角的小方块表示测评成绩是 B 级。

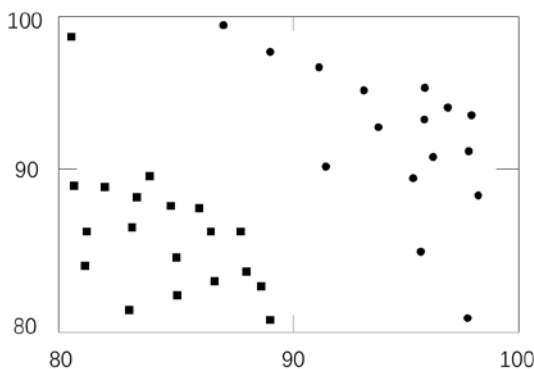


图 21-1 员工的笔试、面试以及测评成绩



当然，公司肯定希望招聘到的员工都是 A 类（表现为 A 级）的。关键是如何根据笔试和面试成绩确定哪些员工可能是未来的 A 类员工呢？或者说，如何根据笔试和面试成绩确保招聘的员工是潜在的优秀员工？偷懒的做法是将笔试和面试的成绩标准都定得较高，但这样做可能会漏掉某些优秀的员工。所以，要合理地确定笔试和面试的成绩标准，确保能够准确高效地招到 A 类员工。例如，在图 21-2 中，分别使用直线对笔试和面试成绩进行了 3 种不同形式的划分，将成绩位于直线左下方的员工划分为 B 类，将成绩位于右上方的员工划分为 A 类。

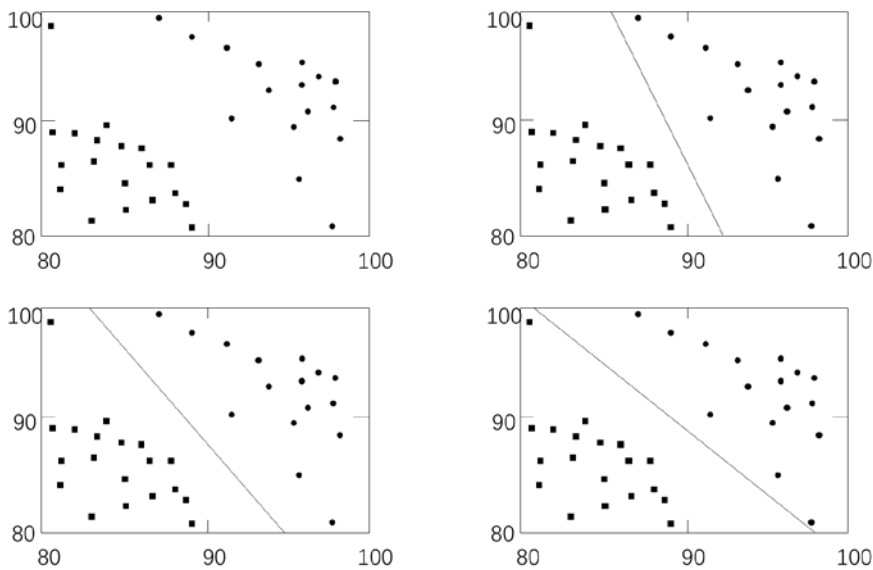


图 21-2 用直线划分数据

2. 分类器

在图 21-2 中用于划分不同类别的直线，就是分类器。在构造分类器时，非常重要的一项工作就是找到最优分类器。

那么，图 21-2 中的三个分类器，哪一个更好呢？从直观上，我们可以发现图 21-2 中右上角的分类器和右下角的分类器都偏向于某一个分类（即与其中一个分类的间距更小），而左下角的分类器实现了“均分”。而左下角的分类器，尽量让两个分类离自己一样远，这样就为每个分类都预留了等量的扩展空间，即使有新的靠近边界的点进来，也能够按照位置划分到对应的分类内。

以上述划分为例，说明如何找到支持向量机：在已有数据中，找到离分类器最近的点，确保它们离分类器尽可能地远。

这里，离分类器最近的点到分类器的距离称为间隔（margin）。我们希望间隔尽可能地大，这样分类器在处理数据时，就会更准确。例如，在图 21-3 中，左下角分类器的间隔最大。

离分类器最近的那些点叫作支持向量（support vector）。正是这些支持向量，决定了分类器所在的位置。

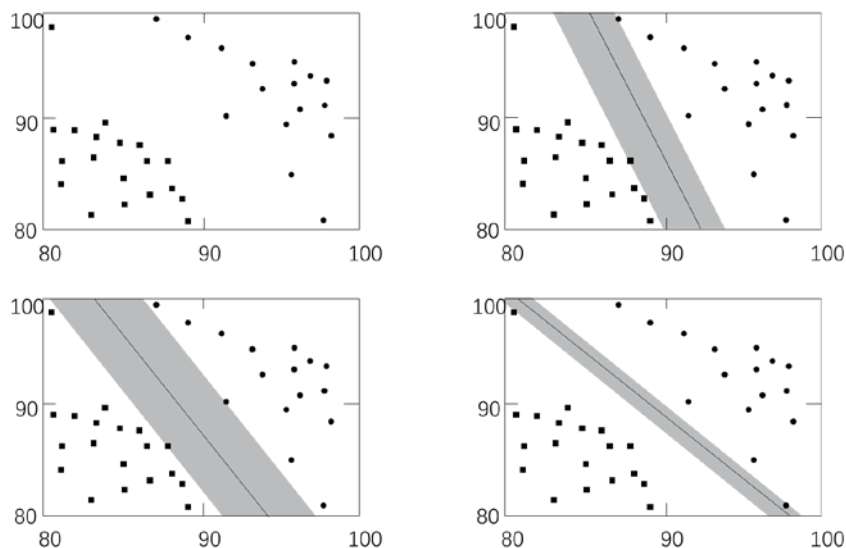


图 21-3 不同分类器的间隔情况

3. 将不可分变为可分

上例的数据非常简单，我们可以使用一条直线（线性分类器）轻易地对其进行划分。而现实中的大多数问题，往往是非常复杂的，不可能像上例一样简单地完成划分。通常情况下，支持向量机会将不那么容易分类的数据通过函数映射变为可分类的。

举个例子，假设我们不小心将豌豆和小米混在了一起。豌豆的个头很大，直径在 10 mm 左右；小米个头小，直径在 1 mm 左右。如果想把它们分开，直接使用直线是不行的。此时，我们可以使用直径为 3 mm 的筛子，将豌豆和小米区分开。在某种意义上，这个筛子就是映射操作，它将豌豆和小米有效地分开了。

支持向量机在处理数据时，如果在低维空间内无法完成分类，就会自动将数据映射到高维空间，使其变为（线性）可分的。简单地讲，就是对当前数据进行函数映射操作。

如图 21-4 所示，在分类时，通过函数 f 的映射，让左图中本来不能用线性分类器分类的数据变为右图中线性可分的数据。当然，在实际操作中，也可能将数据由低维空间向高维空间转换。

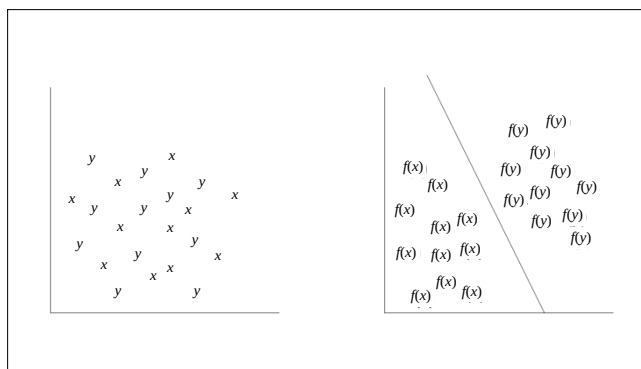


图 21-4 通过函数映射将不可分的数据变为可分的

大家也许会担心，数据由低维空间转换到高维空间后运算量会呈几何级增加，但实际上，支持向量机能够通过核函数有效地降低计算复杂度。

4. 概念总结

尽管上面分析的是二维数据，但实际上支持向量机可以处理任何维度的数据。在不同的维度下，支持向量机都会尽可能寻找类似于二维空间中的直线的线性分类器。例如，在二维空间，支持向量机会寻找一条能够划分当前数据的直线；在三维空间，支持向量机会寻找一个能够划分当前数据的平面（plane）；在更高维的空间，支持向量机会尝试寻找一个能够划分当前数据的超平面（hyperplane）。

一般情况下，把能够可以被一条直线（更一般的情况，即一个超平面）分割的数据称为线性可分的数据，所以超平面是线性分类器。

“支持向量机”是由“支持向量”和“机器”构成的。

- “支持向量”是离分类器最近的那些点，这些点位于最大“间隔”上。通常情况下，分类仅依靠这些点完成，而与其他点无关。
- “机器”指的是分类器。

综上所述，支持向量机是一种基于关键点的分类算法。

21.2 SVM 案例介绍

在使用支持向量机模块时，需要先使用函数 `cv2.ml.SVM_create()` 生成用于后续训练的空分类器模型。该函数的语法格式为：

```
svm = cv2.ml.SVM_create( )
```

获取了空分类器 `svm` 后，针对该模型使用 `svm.train()` 函数对训练数据进行训练，其语法格式为：

```
训练结果 = svm.train(训练数据, 训练数据排列格式, 训练数据的标签)
```

式中参数的含义如下：

- 训练数据：表示原始数据，用来训练分类器。例如，前面讲的招聘的例子中，员工的笔试成绩、面试成绩都是原始的训练数据，可以用来训练支持向量机。
- 训练数据排列格式：原始数据的排列形式有按行排列（`cv2.ml.ROW_SAMPLE`，每一条训练数据占一行）和按列排列（`cv2.ml.COL_SAMPLE`，每一条训练数据占一列）两种形式，根据数据的实际排列情况选择对应的参数即可。
- 训练数据的标签：原始数据的标签。
- 训练结果：训练结果的返回值。

例如，用于训练的数据为 `data`，其对应的标签为 `label`，每一条数据按行排列，对分类器模型 `svm` 进行训练，所使用的语句为：

```
返回值 = svm.train(data, cv2.ml.ROW_SAMPLE, label)
```

完成对分类器的训练后，使用 `svm.predict()` 函数即可使用训练好的分类器模型对测试数据

进行分类，其语法格式为：

```
(返回值, 返回结果) = svm.predict(测试数据)
```

以上是支持向量机模块的基本使用方法。在实际使用中，可能会根据需要对其中的参数进行调整。OpenCV 支持对多个参数的自定义，例如：可以通过 `setType()` 函数设置类别，通过 `setKernel()` 函数设置核类型，通过 `setC()` 函数设置支持向量机的参数 `C`（惩罚系数，即对误差的宽容度，默认值为 0）。

下面通过一个具体案例介绍如何在 Python 中调用 OpenCV 的支持向量机模块进行分类操作。

【例 21.1】 已知老员工的笔试成绩、面试成绩及对应的等级表现，根据新入职员工的笔试成绩、面试成绩预测其可能的表现。

根据题目要求，首先构造一组随机数，并将其划分为两类，然后使用 OpenCV 自带的支持向量机模块完成训练和分类工作，最后将运算结果显示出来。具体步骤如下。

1. 生成模拟数据

首先，模拟生成入职一年后表现为 A 级的员工入职时的笔试和面试成绩。构造 20 组笔试和面试成绩都分布在 [95, 100) 区间的数据对：

```
a = np.random.randint(95,100, (20, 2)).astype(np.float32)
```

上述模拟成绩，在一年后对应的工作表现为 A 级。

接下来，模拟生成入职一年后表现为 B 级的员工入职时的笔试和面试成绩。构造 20 组笔试和面试成绩都分布在 [90, 95) 区间的数据对：

```
b = np.random.randint(90,95, (20, 2)).astype(np.float32)
```

上述模拟成绩，在一年后对应的工作表现为 B 级。

最后，将两组数据合并，并使用 `numpy.array` 对其进行类型转换：

```
data = np.vstack((a,b))
data = np.array(data,dtype='float32')
```

2. 构造分组标签

首先，为对应表现为 A 级的分布在 [95, 100) 区间的数据，构造标签 “0”：

```
aLabel=np.zeros((20,1))
```

接下来，为对应表现为 B 级的分布在 [90, 95) 区间的数据，构造标签 “1”：

```
bLabel=np.ones((20,1))
```

最后，将上述标签合并，并使用 `numpy.array` 对其进行类型转换：

```
label = np.vstack((aLabel, bLabel))
label = np.array(label,dtype='int32')
```

3. 训练

用支持向量机模块对已知的数据和其对应的标签进行训练：

```
svm = cv2.ml.SVM_create()
```



```
result = svm.train(data,cv2.ml.ROW_SAMPLE, label)
```

4. 分类

生成两个随机的数据对(笔试成绩, 面试成绩)用于测试。可以用随机数, 也可以直接指定两个数字。

这里, 我们想观察一下笔试和面试成绩差别较大的数据如何分类。用如下语句生成成绩:

```
test = np.vstack([[98,90],[90,99]])
test = np.array(test,dtype='float32')
```

然后, 使用函数 `svm.predict()` 对随机成绩分类:

```
(p1,p2) = svm.predict(test)
```

5. 显示分类结果

将基础数据 (训练数据)、用于测试的数据 (测试数据) 在图像上显示出来:

```
plt.scatter(a[:,0], a[:,1], 80, 'g', 'o')
plt.scatter(b[:,0], b[:,1], 80, 'b', 's')
plt.scatter(test[:,0], test[:,1], 80, 'r', '*')
plt.show()
```

将测试数据及预测分类结果显示出来:

```
print(test)
print(p2)
```

根据上述分析, 编写程序如下:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 第 1 步 准备数据
# 表现为 A 级的员工的笔试、面试成绩
a = np.random.randint(95,100, (20, 2)).astype(np.float32)
# 表现为 B 级的员工的笔试、面试成绩
b = np.random.randint(90,95, (20, 2)).astype(np.float32)
# 合并数据
data = np.vstack((a,b))
data = np.array(data,dtype='float32')
# 第 2 步 建立分组标签, 0 代表 A 级, 1 代表 B 级
#aLabel 对应着 a 的标签, 为类型 0-等级 A
aLabel=np.zeros((20,1))
#bLabel 对应着 b 的标签, 为类型 1-等级 B
bLabel=np.ones((20,1))
# 合并标签
label = np.vstack((aLabel, bLabel))
label = np.array(label,dtype='int32')
# 第 3 步 训练
# 用 ml 机器学习模块 SVM_create() 创建 svm
svm = cv2.ml.SVM_create()
```

```

# 属性设置, 直接采用默认值即可
#svm.setType(cv2.ml.SVM_C_SVC) # svm type
#svm.setKernel(cv2.ml.SVM_LINEAR) # line
#svm.setC(0.01)
# 训练
result = svm.train(data,cv2.ml.ROW_SAMPLE, label)
# 第 4 步 预测
# 生成两个随机的笔试成绩和面试成绩数据对
test = np.vstack([[98,90],[90,99]])
test = np.array(test, dtype='float32')
# 预测
(p1,p2) = svm.predict(test)
# 第 5 步 观察结果
# 可视化
plt.scatter(a[:,0], a[:,1], 80, 'g', 'o')
plt.scatter(b[:,0], b[:,1], 80, 'b', 's')
plt.scatter(test[:,0], test[:,1], 80, 'r', '*')
plt.show()
# 打印原始测试数据 test, 预测结果
print(test)
print(p2)

```

运行上述程序, 会显示如图 21-5 所示的结果, 图中左下角的方块代表测评成绩为 A 级的员工, 右上角的小圆点代表测评成绩为 B 级的员工, 另外的两个五角星代表需要分类的新入职员工。

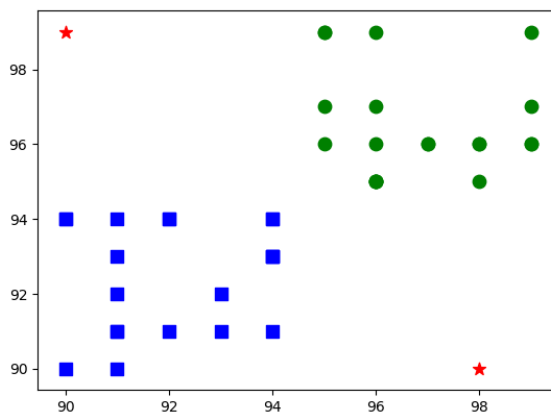


图 21-5 【例 21.1】程序的运行结果

同时, 程序会在控制台输出如下运行结果:

```

[[98. 90.]
 [90. 99.]]
[[1.]
 [1.]]

```

运行结果表明:



- 笔试成绩为 98 分，面试成绩为 90 分，对应的分类为 1，即该员工一年后的测评可能为 B 级（表现良好）。
- 笔试成绩为 90 分，面试成绩为 99 分，对应的分类为 1，即该员工一年后的测评可能为 B 级（表现良好）。

因为我们采用随机方式生成数据，所以每次运行时所生成的数据会有所不同，运行结果也就会有所差异。